
EpiCache: Episodic KV Cache Management for Long-Term Conversation on Resource-Constrained Environments

Minsoo Kim¹ Arnav Kundu¹ Han-Byul Kim¹ Richa Dixit¹ Minsik Cho¹

Abstract

Modern large language models (LLMs) extend context lengths to millions of tokens, enabling coherent, personalized responses grounded in long conversational history. However, the Key-Value (KV) cache grows linearly with the extended dialogue history, causing the model’s memory footprint to quickly exceed device limits. While recent KV cache compression methods attempt to reduce memory usage, most apply cache eviction after processing the entire context, incurring unbounded peak memory usage. Additionally, query-dependent eviction narrows the cache semantics to a single query, leading to failure cases in multi-turn conversations. In this paper, we introduce EPICACHE, a training-free KV cache management framework for long conversational question answering (LongConvQA) under fixed memory budgets. EPICACHE bounds cache growth through block-wise prefill and preserves topic-relevant context via episodic KV compression, which clusters conversation history into coherent episodes and performs episode-specific KV cache eviction. Across three LongConvQA benchmarks (LongMemEval, Realtalk, and LoCoMo), EPICACHE improves accuracy by up to 30%, achieves near-full-cache accuracy under 4–6× compression, and reduces latency and peak memory by up to 2.4× and 3.7×, respectively.

1. Introduction

Large language models (LLMs) (Brown et al., 2020; Yang et al., 2025a; Touvron et al., 2023; Jiang et al., 2023) have significantly extended their maximum context lengths, with LLM-based AI assistants now capable of processing millions of tokens (Reid et al., 2024; Meta, 2025). This capa-

bility enables assistants to leverage extensive dialogue histories when generating responses, producing personalized and contextually coherent outputs (OpenAI, 2024; Anthropic, 2024), which are central requirements for conversational AI applications (Fu et al., 2022).

To benchmark whether assistants can leverage long conversational histories, recent work has formalized Long Conversational Question Answering (LongConvQA) using both human-human conversations and user-AI assistant interactions (Maharana et al., 2024; Lee et al., 2025; Wu et al., 2025a). In this setting, an assistant must answer a sequence of user questions grounded in dialogue histories spanning multiple sessions over days or weeks. Most existing systems tackle LongConvQA by maintaining a retrieval-based memory bank that incrementally summarizes and stores conversation content and retrieves relevant entries for each query (Zhong et al., 2024; Chhikara et al., 2025; Pan et al., 2025; Tan et al., 2025). However, such systems rely on repeated API-based LLM calls to maintain the memory bank and do not bound memory usage at inference time, limiting their applicability to resource-constrained environments.

We study how to enable LongConvQA in resource-constrained environments by designing a Key-Value (KV) cache management framework. The KV cache stores the Key and Value states of each token for reuse in autoregressive generation, but its size grows linearly with context length, creating severe challenges in extended conversations. For instance, in multi-day dialogues between a user and an assistant (Wu et al., 2025a), the KV cache of LLaMA3.2-3B exceeds 7GB after only 30 sessions—larger than the size of the model parameters. This underscores the importance of cache management for deploying conversational AI systems on resource-constrained devices such as smartphones, where memory usage is strictly limited (Alizadeh et al., 2024; Liu et al., 2024b).

Prior work has attempted to mitigate the growing memory usage of KV cache through various compression techniques (Zhang et al., 2023; Li et al., 2024; Cai et al., 2025). Yet two major limitations remain in achieving LongConvQA under resource-constrained environments. First, most methods apply compression after prefill of the entire input context (*post-prefill*), causing peak memory usage that scales lin-

¹Apple. Correspondence to: Minsoo Kim <minsoo@apple.com>, Minsik Cho <minsik@apple.com>.

early with input length. Second, *query-dependent* eviction method (Li et al., 2024) retains cache entries by focusing on the current query and may evict evidence required in later turns, degrading accuracy in multi-turn conversations.

To address these challenges, we propose EpiCache, a training-free KV cache management framework that enforces a constant memory footprint through **block-wise prefill** and **episodic organization**. After processing each block, we evict less critical KV entries to free space for the next block, ensuring memory consumption is bounded. Based on block prefill, EpiCache incorporates an episodic clustering method inspired by conversation segmentation studies (Joty et al., 2013; Galley et al., 2003). Specifically, we apply clustering to group conversation history into coherent episodes, and perform episodic KV cache compression, yielding topic-specific caches while using constrained memory.

Finally, we find that LLMs exhibit different sensitivities to block prefill eviction across layers. Building on this observation, we propose an adaptive layer-wise budget allocation strategy that distributes the KV cache budget proportionally to each layer’s sensitivity. Together with episodic KV cache management, this enables EpiCache to preserve long-term conversation history while operating under strict memory limits, yielding up to 30% higher LongConvQA accuracy than recent baselines and sustaining accuracy comparable to full KV under 4–6× cache compression. In addition, our block-wise cache control framework reduces peak memory usage by up to 3.5×, while cache eviction accelerates decoding, cutting decoding latency by up to 2.4× compared to full KV.

2. Background

We begin by formalizing Long Conversational Question-Answering (LongConvQA) in Section 2.1. We then discuss memory-constrained KV cache management in Section 2.2, comparing post- and block-prefill eviction and discussing the resulting accuracy-memory trade-offs. Finally, in Section 2.3, we review attention-guided cache compression with patched prompts and present analyses that motivate our method, EpiCache.

2.1. Long Conversational QA Formulation

We formalize LongConvQA as answering a sequence of user queries $\mathcal{Q} = \{q_1, \dots, q_{N_q}\}$, with N_q denoting the total number of queries, conditioned on long conversational history (Maharana et al., 2024; Lee et al., 2025; Wu et al., 2025a). Let the dialogue history be represented as an ordered sequence of N_u utterances where each utterance u_j pairs a role r_j with text t_j :

$$\begin{aligned} \mathcal{H} &= \{u_1, u_2, \dots, u_{N_u}\}, \\ u_j &= (r_j, t_j), \quad r_j \in \{\text{speaker}_1, \text{speaker}_2\} \end{aligned} \quad (1)$$

Given a long conversation $\mathcal{H}$, an LLM encodes it into a Key-Value (KV) cache $\text{KV}_{\mathcal{H}}$. For L layers and H KV heads, encoding N tokens produces $L \times H \times N$ KV entries, growing linearly with the conversation length. In this work, we focus on token-level cache compression, where KV entries of less important tokens are evicted; the resulting compressed cache is denoted as $\widetilde{\text{KV}}_{\mathcal{H}} \subseteq \text{KV}_{\mathcal{H}}$, and we use the terms compression and eviction interchangeably.

Our goal is to generate accurate answers for all queries $q_1, \dots, q_{N_q}$ grounded in the dialogue history $\mathcal{H}$, using a compressed cache $\widetilde{\text{KV}}_{\mathcal{H}}$ that satisfies a memory budget M , i.e., with size $L \times H \times M$, while preserving answers comparable to full KV cache ($\text{KV}_{\mathcal{H}}$) based generation:

$$f_{\text{LM}}(q_i | \widetilde{\text{KV}}_{\mathcal{H}}) \approx f_{\text{LM}}(q_i | \text{KV}_{\mathcal{H}}), \quad i = 1, \dots, N_q. \quad (2)$$

This formulation evaluates performance in a *multi-turn conversational* setting, where multiple query–answer pairs are grounded in a shared dialogue history.

2.2. KV Cache Management: Post vs. Block Prefill

Most existing KV compression approaches reduce cache size in the decoding stage by performing eviction *after* the full context has been prefilled, i.e., *Post Prefill Eviction* (Li et al., 2024; Feng et al., 2026; Cai et al., 2025; Kim et al., 2026). As shown in Figure 1a, this design causes peak memory usage to grow linearly with input length, since the entire context must be cached before any eviction takes place. Even with optimized attention kernels (Dao, 2024), memory demand the prefill stage remains unbounded, as observed in Figure 1c top.

To bound memory growth, *Block Prefill Eviction* (Kim et al., 2024; Corallo & Papotti, 2024; Park et al., 2025) processes the input in a block-wise way, handling one segment at a time under a fixed budget M . Each step adds M_{block} tokens, after which an eviction step reduces KV cache entries back to M . For example, in Figure 1b, the budget is $M = 1$, and each block adds $M_{\text{block}} = 3$ tokens that are then evicted back to M . This design ensures that the number of cache entries never exceeds $M + M_{\text{block}}$, keeping peak GPU memory usage flat with input length as highlighted in Figure 1c top.

However, this bounded memory comes with a steep accuracy trade-off: when the state-of-the-art post prefill eviction method KVzip, (Kim et al., 2026; NVIDIA, 2025) is applied in the block prefill setting, LongConvQA scores (Maharana et al., 2024) degrade sharply across all budget levels. This underscores a central challenge—while block prefill guarantees constant memory usage, adapting post-prefill eviction methods to this setting severely undermines answer quality in LongConvQA.

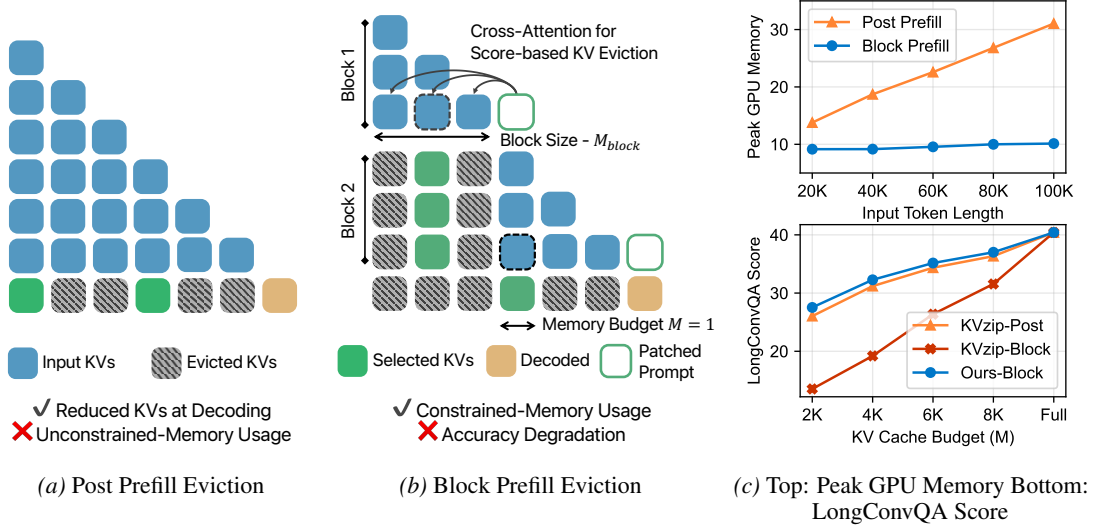


Figure 1. KV Cache Management Analysis. (a) Post prefill eviction: eviction after full-context prefill, reducing KV size at decoding but causing unbounded memory usage. (b) Block prefill eviction: input processed in 3-token blocks with patched prompts for scoring, then evicted down to 1 token. (c) Top: Peak GPU memory vs. input length on LLaMA-3.2-3B with A100. Bottom: LongConvQA accuracy of KV compression methods under post vs. block prefill on LLaMA-3.2-3B.

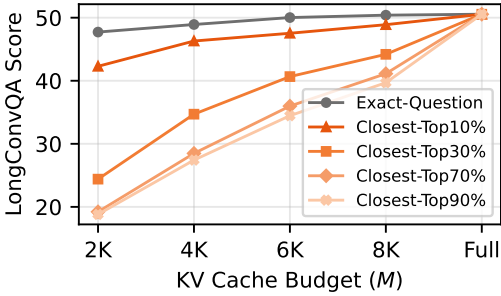


Figure 2. Patched-Prompt Analysis: LoCoMo results with LLaMA3.1-8B under block prefill. Patched prompts are formed by selecting the top 10%–90% similar conversation utterances to q_i .

2.3. Attention-guided KV Cache Compression

To address the accuracy degradation of block prefill eviction, prior work employs attention-based token scoring with a patched prompt. Here, token importance is quantified by the cross-attention it receives from query tokens: $\text{Attn}(x_t \rightarrow x_i)$ which denotes the attention weight from a query token x_t to a key token x_i . Tokens that receive higher attention from queries are considered more important, while those with lower scores are evicted to satisfy the memory budget M .

To provide guidance for cache eviction, the *patched prompt* strategy (Kim et al., 2024; Corallo & Papotti, 2024) appends an auxiliary prompt of length p after each block ending at token n . These queries $x_{n+1}, \dots, x_{n+p}$ attend back to the preceding block tokens $x_1, \dots, x_n$, as illustrated in Figure 1b. The resulting importance score s_i of token i is aggregated by taking the maximum across token-axis (Kim

et al., 2026), as defined in Equation (3). The patched prompt is only used for scoring and not retained in the KV cache.

$$s_i^{\max} = \max_{t \in [n+1, n+p]} \text{Attn}(x_t \rightarrow x_i). \quad (3)$$

In block prefill, accuracy strongly depends on the content of the patched prompt. To quantify this effect, Figure 2 presents a controlled experiment that assumes oracle access to the future user query and inserts it directly as the patched prompt (Exact-Question), yielding the highest accuracy under a fixed memory budget. While this setting provides an upper bound on performance, it is infeasible in LongConvQA for two reasons (Kim et al., 2026): (i) future user queries are unknown at compression time, and (ii) such *query-dependent* compression requires re-prefilling the entire dialogue history for every new question.

Since the dialogue history $\mathcal{H}$ consists of query-answer turns, it offers an opportunity to approximate the future query with semantically related turns: we embed the future queries and all utterances $u_1, \dots, u_{N_u}$ into a shared embedding space, rank utterances by semantic similarity, and construct the patched prompt from the top-ranked ones. Using the resulting patched-prompt, we run block prefill eviction and evaluate LongConvQA accuracy. As shown in Figure 2, the prompts formed from the most semantically aligned utterances (top 10%) nearly match Exact-Question performance, while accuracy degrades as semantic alignment decreases. This observation motivates our goal: identifying, without access to future queries, patched prompts that approximate unseen questions. To this end, we introduce an unsupervised clustering method to discover dialogue segments aligned with future queries (Section 3.1).

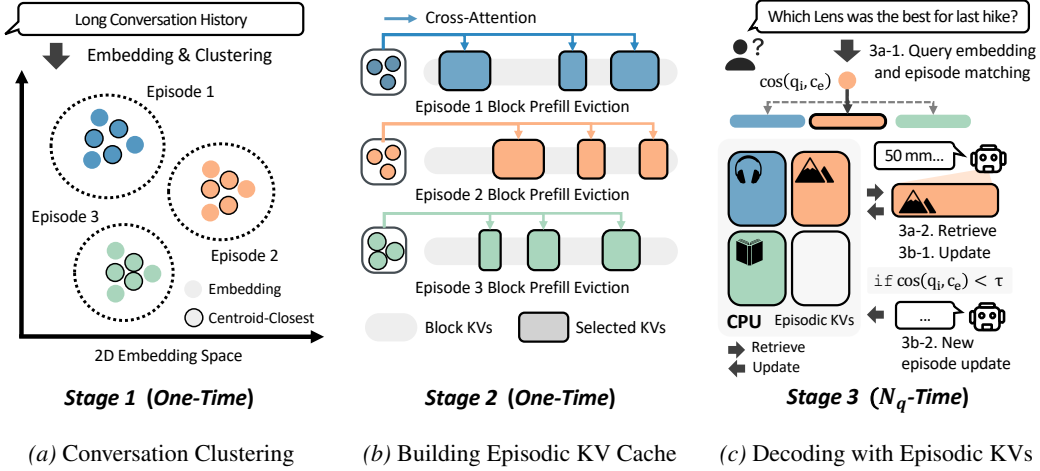


Figure 3. **EpiCache Overview.** (a) segmentation and embedding of the conversation, followed by clustering into episodes. (b) building episodic KV caches under a fixed memory usage based on representative segments of each episode. (c) an incoming query is embedded, matched to the closest episode, and the corresponding cache is retrieved for answer generation, and updated with newly generated tokens.

3. Method

3.1. Episodic KV Cache Management

Conversations span multiple topics within a single session (Maharana et al., 2024; Pan et al., 2025). Such long histories can be segmented into coherent episodes, where subsequent utterances are grounded in different episodes of the prior dialogue. This motivates EpiCache: by clustering long-term conversation histories into multiple episodes and constructing episode-specific caches, we match each incoming query to the most relevant cache for accurate answer generation. As illustrated in Figure 3, this process unfolds in three stages: offline conversation clustering, episodic KV cache construction, and query-matched decoding.

Stage 1. Conversation Clustering. (Figure 3a) For clustering conversation, we first divide the raw dialogue history $\mathcal{H}$ into segments of w_{embed} utterances, denoted as $\mathcal{H} = S_1, \dots, S_K$.

$$S_k = u_{(k-1)w_{\text{embed}}+1}, \dots, u_{\min(kw_{\text{embed}}, N_u)}, \quad k = 1, \dots, K, \quad K = \left\lceil \frac{N_u}{w_{\text{embed}}} \right\rceil \quad (4)$$

Each segment S_k is encoded with a lightweight encoders (Zhang et al., 2025) f_{embed} into a vector embedding $\mathbf{e}_k \in \mathbb{R}^d$, capturing the segment’s semantics. We then apply K-Means clustering $\mathcal{C}(\cdot)$ to the embeddings $\{\mathbf{e}_k\}_{k=1}^K$:

$$\mathcal{C}(\{\mathbf{e}_k\}_{k=1}^K) \rightarrow \{\mathcal{E}_1, \dots, \mathcal{E}_E\}, \quad \bigcup_{e=1}^E \mathcal{E}_e = \{S_1, \dots, S_K\}. \quad (5)$$

Equation (5) partitions $\mathcal{H}$ into E semantically coherent topical episodes (Raedt et al., 2024).¹ For each cluster $\mathcal{E}_e$, we

can compute its centroid embedding $\mathbf{c}_e$:

$$\mathbf{c}_e = \frac{1}{|\mathcal{E}_e|} \sum_{S_k \in \mathcal{E}_e} \mathbf{e}_k, \quad S_{\text{centroid-closest}}^{(e)} = \arg \max_{S_k \in \mathcal{E}_e} \cos(\mathbf{e}_k, \mathbf{c}_e). \quad (6)$$

We then identify the *representative* segments of the cluster $S_{\text{centroid-closest}}^{(e)}$ —i.e., the conversation segment in each cluster whose embedding is closest to the centroid in terms of cosine similarity. These centroid-closest segments contain multiple turns from both speakers and is used as the patched prompt in the subsequent block prefill eviction step.

Stage 2. Episodic KV Cache Construction. (Figure 3b)

As discussed in Section 2.3, patched prompts guide cache eviction toward retaining tokens relevant to the prompt content. Building on this insight, EpiCache uses the centroid-closest segments of each episode as the patched prompt, thereby collecting episode-specific KV entries under the memory budget M .

For each episode $e \in \{1, \dots, E\}$, we perform block prefill eviction over the entire context, appending its patched prompt after each block. Attention scores are then computed as in Eq. (3), and the top M tokens are retained to form an episode-specific cache $C_{\text{KV}}^{(e)}$. Finally, all episodic caches are collected into $\mathbb{B} = \{C_{\text{KV}}^{(1)}, \dots, C_{\text{KV}}^{(E)}\}$ and stored offline for later retrieval.

Stage 3a. KV Matching and Decoding. (Figure 3c)

At decoding time, each user query q_i is embedded with the same encoder f_{embed} used in clustering, ensuring that it lies in the same representation space as the episode centroids. The query is then matched to the closest centroid as follows:

$$\mathbf{q}_i = f_{\text{embed}}(q_i), \quad e^\dagger = \arg \max_{e \in [1, E]} \cos(\mathbf{q}_i, \mathbf{c}_e). \quad (7)$$

¹Qualitative analysis of clustering is provided in Appendix B.1.

As illustrated in (3a-2) of Figure 3c, the framework retrieves the corresponding episodic KV cache $C_{KV}^{(e^\dagger)}$ from $\mathbb{B}$ and conditions generation on it: $f_{LM}(q_i | C_{KV}^{(e^\dagger)})$. This design enables query-specific retrieval while keeping cache size bounded under memory budget M .

Stage 3b. Episodic KV Cache Update. (Figure 3c) For each turn, EPICACHE manages episodic KV caches in two ways to support a continuous conversational experience.

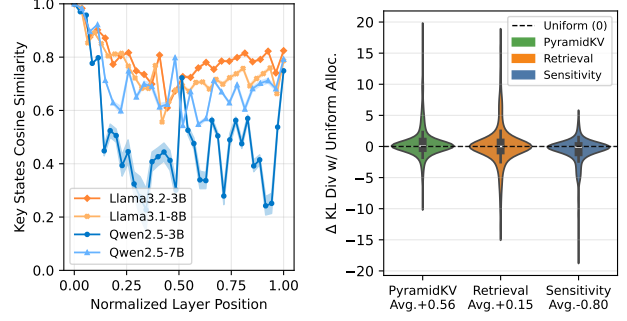
- **Update an existing episode as in (3b-1) of Figure 3c):** When an episodic KV cache is retrieved for decoding as in Stage 3a, the KV pairs generated at the current turn are appended to the retrieved episodic cache, enriching episode-specific memory. If a subsequent turn is matched to a different episode due to a topic shift, the episodic cache used in the previous turn—appended with newly generated KV pairs—is written back to the episodic KV cache set.
- **Create a new episode as in (3b-2) of Figure 3c):** If the query embedding is not sufficiently similar to any existing centroid ($\max_e \cos(\mathbf{q}_i, \mathbf{c}_e) < \tau$), EPICACHE creates a new episode. A new episodic KV cache is initialized from the KV pairs of the new turn and added to the cache set $\mathbb{B}$, allowing previously unseen topics to be incorporated into the episodic structure.

To maintain strict memory bounds, EPICACHE refreshes an episodic cache only when its size exceeds $M + M_{\text{block}}$. Upon reaching this cache budget, we collect the newly accumulated episode history $\mathcal{H}'$ and re-run the same clustering procedure as in Stage 1. We then update the representative segment for the episode, and rebuild the episodic caches by block prefill eviction guided by the updated prompt under the fixed budget M . An interesting extension is to update the compressed episodic KV cache using only newly added turns, which we leave as future work.

3.2. Sensitivity-aware Layer-wise KV Budget Allocation

We further address the accuracy degradation of block prefill by proposing a KV cache *budget allocation* strategy. The key idea is to measure how much each layer’s Key states representation deviates under block prefill and to distribute a KV budget across layers in proportion to this deviation.

Simulating Block Prefill via Custom Masking. To quantify the deviation caused by block prefill eviction, we introduce a custom masking scheme. Each transformer layer is represented as a function f that takes the previous layer’s output $X^{(\ell-1)} \in \mathbb{R}^{N \times d}$. Here, N denotes the input sequence length and d the hidden dimension. The function produces the ℓ -th layer output $X^\ell = f(X^{\ell-1}, \mathcal{M})$, where $\mathcal{M}$ is the standard causal mask.



(a) Key states cosine similarity (b) Δ KL divergence w/ full KV

Figure 4. Layer-wise Sensitivity Analysis. (a) Key states cosine similarity across layer positions. (b) KL divergence measured between block prefill ($M=4K$) and full KV answer predictions, with uniform allocation as the baseline. Qwen2.5-7B used.

We replace $\mathcal{M}$ with a custom mask $\mathcal{M}'$ that enforces a budget M , attending to sink tokens and the most recent tokens (Xiao et al., 2024). This design follows static KV cache compression methods, allowing us to simulate block prefill eviction in a single forward pass and directly measure its effect on layer representations.

Layer Sensitivity Guided KV Budget Allocation. We quantify the per-layer impact of block prefill eviction by comparing Key states² produced under the causal mask $\mathcal{M}$ and the custom mask $\mathcal{M}'$. For each layer ℓ , the forward pass under each mask produces:

$$\begin{aligned} K_{\text{full}}^\ell &= f(X_{\text{full}}^{\ell-1}, \mathcal{M}) W_K^\ell, \\ K_{\text{block}}^\ell &= f(X_{\text{block}}^{\ell-1}, \mathcal{M}') W_K^\ell \end{aligned} \quad (8)$$

where K_{full}^ℓ and K_{block}^ℓ are the ℓ -th layer Key states computed under $\mathcal{M}$ and $\mathcal{M}'$, respectively. We then define layer sensitivity as the average cosine similarity between the two sets of Key vectors across attention heads and input tokens:

$$\sigma_\ell = \frac{1}{HN} \sum_{h=1}^H \sum_{i=1}^N \cos(k_{\text{full},i}^{(\ell,h)}, k_{\text{block},i}^{(\ell,h)}) \quad (9)$$

Empirically, σ_ℓ exhibits large variation across layers yet remains consistent across different inputs in Figure 4a (shaded regions denote input variance), indicating that layer sensitivity is *model-dependent* rather than input-dependent. We define $s_\ell = 1 - \sigma_\ell$ as the sensitivity score for layer ℓ . Based on this observation, we propose a sensitivity-aware budget allocation strategy that assigns larger cache budgets to layers more sensitive to block prefill and smaller budgets to less sensitive ones. Specifically, we redistribute the global budget ($M \cdot L$) according to layer sensitivity scores s_ℓ , with α controlling how sharply the allocation emphasizes

²Further details regarding the rationale for using Key states deviation are provided in Appendix B.2.

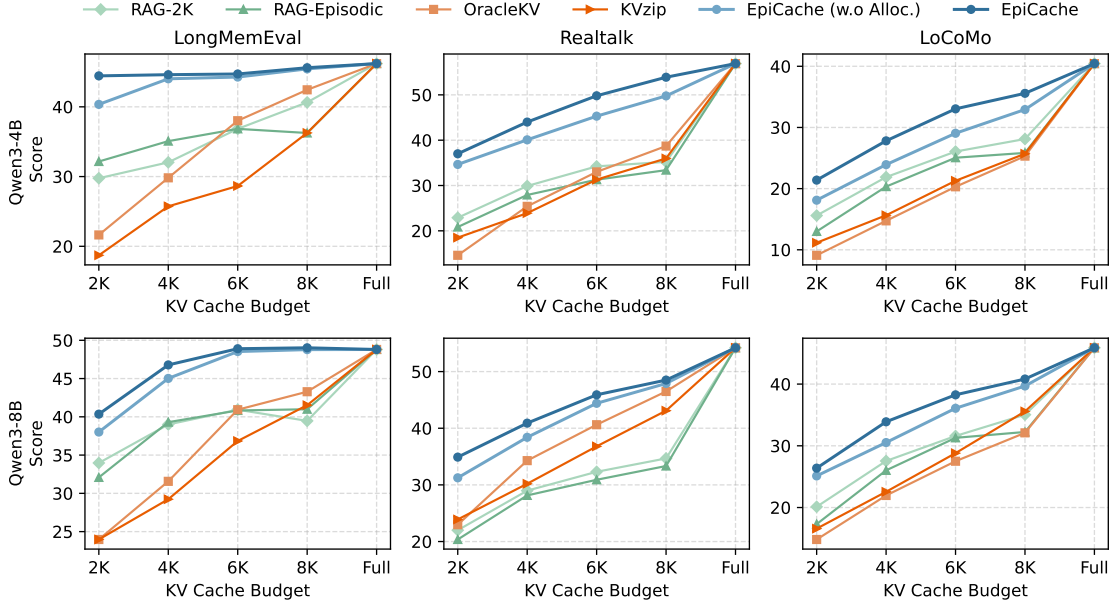


Figure 5. **LongConvQA Evaluation** (LongMemEval, Realtalk, and LoCoMo) results with fixed KV cache budgets $M \in \{2K, 4K, 6K, 8K\}$ across Qwen3 family models. The number of episodes (clusters) fixed to $E=4$ in all experiments.

sensitive layer:

$$M_{\ell}^{\text{alloc}} = \frac{s_{\ell}^{\alpha}}{\sum_{j=1}^L s_j^{\alpha}} \cdot (L \cdot M), \quad \sum_{\ell=1}^L M_{\ell}^{\text{alloc}} = L \cdot M \quad (10)$$

We evaluate the sensitivity-aware approach by measuring how budget allocation shifts the KL divergence between answer predictions generated from block prefill eviction and those generated from the full KV cache, where negative values indicate closer alignment with full KV answer generation. As shown in Figure 4b, sensitivity-aware allocation shifts KL divergence by -0.80 relative to uniform allocation. In contrast, recent budget allocation methods such as PyramidKV (Cai et al., 2025), which follows a pyramid-shaped budgeting, and retrieval head profiling based allocation (Wu et al., 2025b) tend to increase KL divergence. Our reduced KL divergence is reflected in improved LongConvQA accuracy (see Figure 5), as further discussed in Section 4.1.

4. Experiments

Models and Benchmarks. We evaluate on a comprehensive family of models: Qwen3 (Yang et al., 2025a), LLaMA3.1 and 3.2 (Grattafiori et al., 2024) and Qwen2.5 (Qwen et al., 2025). All evaluations follow the LongConvQA formulation in Equation (2), where models answer various queries grounded in long conversation histories. We use three benchmarks: Realtalk (Lee et al., 2025) and LoCoMo (Maharana et al., 2024), containing multi-day human-human dialogues, and LongMemEval (Wu et al., 2025a), consisting of multi-session user-LLM dialogues. Further details of benchmarks are provided in Section A.1.

Baselines. (1) RAG: As an alternative memory-constrained design for LongConvQA, we consider retrieval-augmented generation (Lewis et al., 2020) that retrieve raw-text from the dialogue history and construct the KV cache for each query. RAG-2K retrieves relevant 2K-token chunks, while RAG-Episodic clusters the history into episodes (Section 3.1) and retrieves segments from the most relevant episode under the cache budget M .

(2) KV Cache Compression: We compare different compression strategies *within* the block prefill eviction framework under the constrained memory budgets. We include recent query-agnostic compression methods, KVzip (Kim et al., 2026) and OracleKV (Zhu et al., 2025), and additionally compare representative KV cache compression approaches StreamingLLM (Xiao et al., 2024), SnapKV (Li et al., 2024), KeyDiff (Park et al., 2025), and InfiniPot (Kim et al., 2024). Detailed dataset information, baseline setups, and EPICACHE configurations are provided in Appendix A.

4.1. Main Results

Figure 5 presents the main LongConvQA results on three benchmarks (LongMemEval, Realtalk, and LoCoMo) across different KV cache budgets with Qwen3-4B and 8B models. EPICACHE consistently achieves the best performance across all cache budgets and benchmarks, outperforming both retrieval-based baselines (RAG-2K and RAG-Episodic) and recent KV cache compression baselines (KVzip and OracleKV) by wide margins, especially with low cache budgets (2-4K). Moreover, comparing EPICACHE with and without sensitivity-aware budget allocation (EPICACHE w.o. alloc.) shows that our allocation strategy provides a consistent ac-

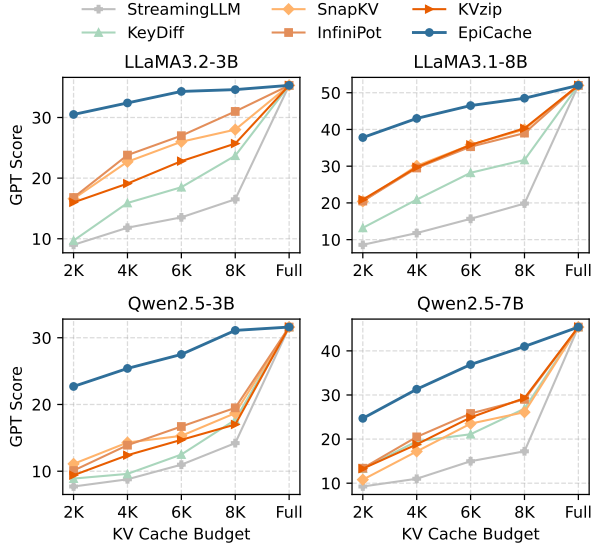


Figure 6. **Evaluation with Various Models** results with Realtalk. Evaluated with fixed KV cache budgets $M \in \{2K, 4K, 6K, 8K\}$ across four LLMs. The number of episodes (clusters) fixed to $E=4$ in all experiments. See Figure A4 for other two benchmarks.

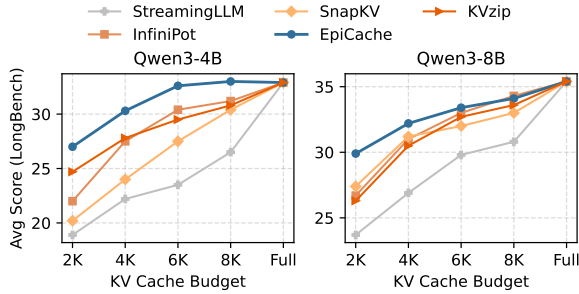


Figure 7. **Long Form Document QA Results** on NarrativeQA, HotpotQA, MuSiQue, and QMSum (averaged). Evaluated with fixed KV cache budgets $M \in \{2K, 4K, 6K, 8K\}$ on Qwen3-4B and Qwen3-8B. The number of episodes fixed to $E=4$.

curacy gain across models and cache budgets. To further validate generality, Figure 6 reports results on Realtalk with four different LLMs, where EPICACHE substantially outperforms a broader set of KV cache compression methods (StreamingLLM, SnapKV, KeyDiff, and InfiniPot), yielding up to 30 absolute score improvement under constrained budgets, while converging to full KV performance as the budget increases.

Figure 7 probes whether the core mechanism of EPICACHE—clustering semantically coherent segments and retrieving the most relevant episode at inference—transfers beyond dialogue to unstructured document contexts, using four long-form document QA tasks from LongBench (Bai et al., 2024) (NarrativeQA, HotpotQA, MuSiQue, and QMSum; average input length 15K–36K tokens). EPICACHE consistently outperforms all baselines across both Qwen3-4B and 8B, with the largest gains observed under constrained

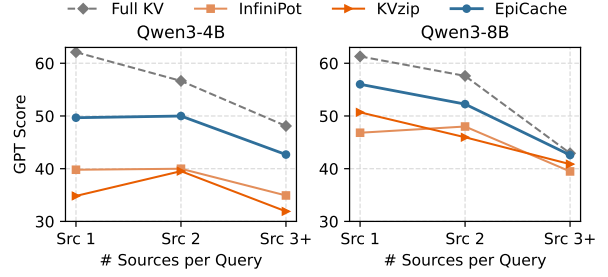


Figure 8. **Multi-Episode Analysis** on Realtalk with Qwen3-4B and 8B at budget $M=8K$. Queries are grouped by the number of sources required to answer (*Src 1/2/3+*), where larger groups demand evidence distributed across more episodes.

Qwen3-4B		Realtalk			LoCoMo		
Method	M	Single	Cross	Temp.	Single	Cross	Temp.
Full KV	-	60.3	55.4	61.7	34.1	32.8	32.6
RAG-2K	4K	31.8	27.9	17.6	24.2	20.0	5.5
RAG-2K	8K	38.4	31.9	22.4	31.6	27.8	7.3
KVzip	4K	26.7	21.8	22.8	13.7	15.7	13.7
KVzip	8K	37.5	35.9	35.0	24.5	24.3	23.0
EPICACHE	4K	45.3	40.3	41.5	29.8	25.8	22.3
EPICACHE	8K	54.6	50.8	55.7	33.3	32.7	29.2

Table 1. **Cross Episode Analysis** on Realtalk and LoCoMo with Qwen3-4B under KV cache budgets $M \in \{4K, 8K\}$.

budgets (2–4K), confirming that episodic KV cache management generalizes effectively to document-level contexts beyond multi-turn conversational structure.

4.2. EPICACHE Analysis

Multi-Episodic Reasoning. We evaluate EPICACHE’s ability to handle queries whose evidence spans multiple conversational episodes from two complementary perspectives. Figure 8 leverages the ground-truth indices of the conversation segments provided by Realtalk (Lee et al., 2025)—which indicate the specific multi-turn segments required to answer each query—to regroup queries by the number of source segments referenced (*Src 1/2/3+*), where larger groups demand evidence distributed across more distinct episodes. Table 1 takes an orthogonal view by categorizing each query as *Single*, *Cross*, or *Temporal* based on the sharpness of its cosine similarity distribution over episode centroids obtained from our clustering stage: queries whose similarity is dominated by a single centroid are labeled *Single*, while those with comparable similarity scores across multiple centroids are labeled *Cross*; Temporal queries, originally defined in Realtalk, require temporal cues that frequently span episode boundaries.

Both analyses consistently show that EPICACHE maintains strong performance as evidence becomes more distributed across episodes, whereas baselines degrade sharply. In Figure 8, EPICACHE consistently stays closer to Full KV ac-

Qwen3-4B ($M=8K$)	Multi-Hop	Temporal	Common	Avg
Full KV	53.6	61.7	52.2	56.9
RAG-Episodic	42.3	22.4	41.0	33.4
KVzip	34.4	35.0	43.3	36.0
<i>Conversation segmentation unit ($E = 4$, w.o budget allocation)</i>				
Words	44.4	50.3	47.6	47.5
Tokens	47.9	47.9	49.3	47.9
Utterances	48.5	51.5	48.1	49.8
<i>Embedding model f_{embed} ($E = 4$, w.o budget allocation)</i>				
LLM-embedding	40.8	43.1	49.1	43.0
MiniLM-L6-v2	45.5	47.9	52.2	47.6
Qwen3-Emb-0.6B	48.5	51.5	48.1	49.8
Qwen3-Emb-4B	48.6	53.1	48.8	50.6
<i>The number of episodes E (w.o budget allocation)</i>				
$E = 2$	46.5	50.0	46.9	47.9
$E = 4$	48.5	51.5	48.1	49.8
$E = 6$	48.0	56.1	50.8	51.1
$E = 8$	48.5	53.4	53.1	51.3
<i>Layer-wise budget allocation (sharpness hyper-parameter α)</i>				
$\alpha = 1$	49.0	55.3	54.4	53.0
$\alpha = 2$	51.7	55.7	54.7	53.9
$\alpha = 4$	50.2	57.4	54.1	53.9
$\alpha = 8$	48.3	52.1	47.3	49.8

Table 2. Ablation Study on the RealTalk benchmark with Qwen3-4B 8K cache budget. LLM-embedding uses the target LLM’s embedding layer as the encoder, and Qwen3-Emb. denotes the Qwen3 embedding model. Highlighted rows indicate the final selected configuration of EpiCache.

curacy across all source segment groups (Src 1 to Src 3+), demonstrating robustness to multi-episode evidence that holds across both Qwen3-4B and 8B. [Table 1](#) further confirms this at the query-type level: on RealTalk at 8K, EpiCache achieves 50.8 / 55.7 on Cross and Temporal queries versus RAG’s 31.9 / 22.4. This robustness stems from a fundamental difference in how context is retained: retrieval-based methods concatenate a small set of retrieved segments at query time, inherently missing dependencies distributed across the full dialogue, while EpiCache computes each episodic KV cache via block-wise prefill over the *entire* conversation history before compression—preserving globally contextualized representations within every episodic cache. Even when evidence spans multiple episodes, EpiCache can surface cross-episode dependencies through these context-aware KV states, rather than relying on retrieval to explicitly identify and assemble relevant segments.

Ablation Study. [Table 2](#) ablates design choices of EpiCache on Realtalk. We first compare conversation segmentation units : utterance-level splitting consistently outperforms word- and token-level alternatives, confirming that breaking natural speaker-turn boundaries degrades final accuracy. For embedding model, we compare the LLM’s embedding layer, a sentence encoder (MiniLM), and Qwen3-Embedding ([Zhang et al., 2025](#)). Qwen3-Embedding achieves the best accuracy, while scaling the

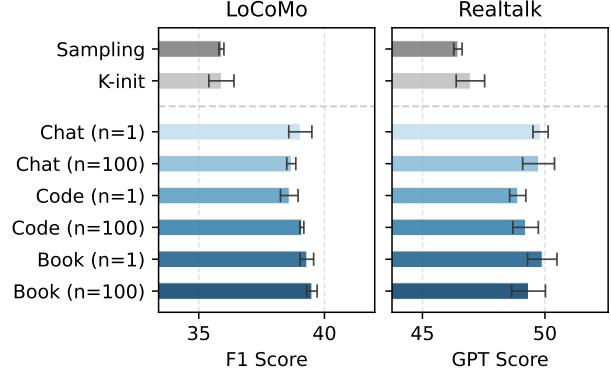


Figure 9. Robustness of Layer-wise Budget Allocation on LoCoMo and Realtalk with Qwen3-8B at budget $M=6K$. Error bars denote $\pm$ std over 5 seeds. *Top*: variance due to K-Means initialization (K-init) and generation sampling, evaluated without budget allocation. *Bottom*: variance across layer sensitivity profiling calibration domain and number of calibration samples ($n=1$ or 100).

encoder to 4B yields only marginal gains, indicating that a lightweight encoder (0.6B) is sufficient with negligible overhead relative to the base LLM. We next sweep the number of episodes E and the sharpness parameter α for layer-wise budget allocation. Increasing E improves performance, but even a small number ($E = 4$) delivers strong accuracy (49.8 vs. 36.0 for KVzip), suggesting that cross-episodic memory is effectively captured in the contextualized KV cache. For sharpness, larger values lead to increasingly skewed layer-wise budgets; moderate sharpness (2–4) performs best, while overly sharp allocation (8) hurts accuracy.

Robustness of Budget Allocation. [Figure 9](#) examines EpiCache’s robustness across two axes: stochasticity (K-Means initialization and generation sampling) and calibration sensitivity (domain choice and sample count for layer-wise budget profiling). Across both LoCoMo and Realtalk, all conditions yield consistent performance with standard deviations below 0.5, confirming that neither clustering randomness nor generation sampling introduces meaningful instability. For calibration, we swept three domains—dialogue history (SAMsum ([Gliwa et al., 2019](#))), code repository (RepoBench-P ([Liu et al., 2024a](#))), and long-form narrative (BookSum ([Kryscinski et al., 2022](#)))—with $n \in \{1, 100\}$ samples each. Profiling with a single sample matches the performance of using 100 samples across all domains, demonstrating that layer-wise sensitivity is largely model-dependent rather than data-dependent: a single offline calibration suffices regardless of domain.

4.3. Efficiency Analysis

[Table 3](#) reports an efficiency analysis aligned with the pipeline in [Figure 3](#), measuring the latency of stages 1–3 of EpiCache and comparing it against two Full KV baselines in a long-term conversational setting under a single-batch

LLaMA3.2-3B Method	300 Turns w. 90K History	Stage 1		Stage 2	Stage 3			Total	QA	Peak
		Embed	Cluster	Prefill	Match	Retr. & wb	Decode	(Per-Turn)	Acc.	Mem. (GB)
Full KV	Latency (sec) # of Execution	0.0 0	0.0 0	27.8 $\times 300$	0.0 0	0.0 0	3.5 $\times 300$	9339.0 (31.1)	46.2	36.3
Full KV w. Prefix Caching	Latency (sec) # of Execution	0.0 0	0.0 0	27.8 $\times 1$	0.0 0	0.0 0	3.5 $\times 300$	1062.8 (3.5)	46.2	36.3
EPICACHE	Latency (sec) # of Execution	25.6 $\times 1$	0.76 $\times 1$	18.0 $\times 4$	0.04 $\times 1$	0.30 $\times 90$	1.4 $\times 300$	545.4 (1.8)	45.6	9.6

Table 3. **Efficiency Analysis** with GPU memory usage (GB) and Latency (sec), under 30 days of 90K-token conversation history followed by subsequent 300 multi-turn conversation with LLaMA3.2-3B. $M = 8K$ cache budget used for EPICACHE. Retr. & wb (retrieval and write-back) stage is executed 90 times over 300 turns (30% topic-change rate), and its cost is amortized accordingly.

inference regime to reflect device-oriented usage. All experiments use a 90K-token dialogue history, corresponding to approximately 30 days of conversational sessions from LongMemEval (Wu et al., 2025a). Conditioned on this history, we evaluate peak GPU memory usage and total latency over a subsequent 300 user–LLM interactions, and report the latency of each sub-stage along with its execution count.

EPICACHE reduces peak GPU memory by nearly $4\times$ (36.3 vs. 9.6), enabling long-term conversational inference under strict memory budgets in mobile environments (Liu et al., 2024b). For latency, the total cost is largely determined by how often each stage is executed. Without caching, Full KV re-prefills the entire history at every turn, resulting in a prohibitively high total latency (9,339 s). While prefix caching (Zheng et al., 2024) removes repeated prefill, decoding still operates over the full KV cache at each turn, leading to a high latency dominated by decoding (1,062.8 s).

In contrast, although EPICACHE incurs one-time costs for conversation clustering and episodic KV cache construction (stage 1-2 of Table 3), EPICACHE enables decoding to operate on a compressed KV cache at every turn, achieving up to $2.4\times$ faster decoding, reducing the total latency over long interactions. EPICACHE achieves a total latency of 545.4 s, yielding nearly an $18\times$ speedup over Full KV and an approximately $2\times$ speedup over prefix caching, while maintaining comparable QA accuracy (46.2 vs. 45.6).

5. Related Work

Prior KV cache eviction methods rely on attention scores to estimate token importance. H2O (Zhang et al., 2023) aggregates entire attention scores for eviction, while SnapKV (Li et al., 2024) uses query-dependent attention scores. InfiniPot (Kim et al., 2024) and OracleKV (Zhu et al., 2025) use designed patched prompts that approximate future queries to derive importance scores. Alongside attention-based eviction, TRIM-KV (Bui et al., 2026) learns token importance via a lightweight retention gate fine-tuned through distillation, and Cartridge (Eyuboglu et al., 2026) trains a corpus-specific KV cache offline using a self-study objec-

tive, both requiring additional training. FlowKV (Liu et al., 2025b) introduces a multi-turn isolation mechanism that avoids recompressing past-turn KV caches, but does not compress conversation history.

Another line of work improves decoding efficiency by selectively retrieving the most relevant parts of the KV cache for each query tokens, thereby reducing the cost of attention computation. Quest (Tang et al., 2024) and ArkVale (Chen et al., 2024) retrieves KV entries at the granularity of pages, while SqueezedAttention (Hooper et al., 2025) and ClusterKV (Liu et al., 2025a) clusters Key states and loads the cluster relevant to the query. IceCache (Mao et al., 2026) further extends this line by combining semantic token clustering with PagedAttention to hierarchically organize KV cache. These methods share two key limitations. First, they operate in the *post-prefill* regime, assuming unconstrained memory usage during prefill. Second, their retrieval units (pages and cluster) do not align with the episodic structure of conversations, limiting their applicability to LongConvQA under strict memory budgets.

6. Conclusion

EPICACHE is the first framework that combines block-wise prefill with episodic clustering and sensitivity-aware budget allocation to preserve topic-relevant context under a fixed memory budget. Across multiple LongConvQA benchmarks, EPICACHE substantially outperforms existing compression methods, demonstrating that long-term interaction is feasible under strict resource constraints and marking a practical step toward on-device conversational AI.

Future Work. Future work includes improving episode construction through more effective conversational embeddings (Yang et al., 2025b) and adaptive episode count selection (Raedt et al., 2024). Evaluating EPICACHE on more challenging benchmarks that require tracking implicit user preferences and handling information updates over time (Jiang et al., 2025; Uddin et al., 2026) presents a natural next step toward long-term conversational agents.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Alizadeh, K., Mirzadeh, S. I., Belenko, D., Khatamifard, S., Cho, M., Del Mundo, C. C., Rastegari, M., and Farajtabar, M. LLM in a flash: Efficient large language model inference with limited memory. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12562–12584, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.678. URL <https://aclanthology.org/2024.acl-long.678/>.
- Anthropic. Introducing the next generation of claude. <https://www.anthropic.com/news/claude-3-family>, 2024.
- Arthur, D. and Vassilvitskii, S. k-means++: the advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA ’07, pp. 1027–1035, USA, 2007. Society for Industrial and Applied Mathematics. ISBN 9780898716245.
- Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., and Li, J. LongBench: A bilingual, multitask benchmark for long context understanding. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3119–3137, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.172. URL <https://aclanthology.org/2024.acl-long.172/>.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf.
- Bui, N., Sharma, S., Lamba, S., Mishra, S., and Ying, R. Cache what lasts: Token retention for memory-bounded KV cache in LLMs. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=qCq3jGb0S>.
- Cai, Z., Zhang, Y., Gao, B., Liu, Y., Li, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Hu, J., and Xiao, W. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=ayi7qezU87>.
- Chen, R., Wang, Z., Cao, B., Wu, T., Zheng, S., Li, X., Wei, X., Yan, S., Li, M., and Liang, Y. Arkvale: Efficient generative LLM inference with recallable key-value eviction. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=4oAt5L4lYe>.
- Chhikara, P., Khant, D., Aryan, S., Singh, T., and Yadav, D. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Corallo, G. and Papotti, P. FINCH: Prompt-guided key-value cache compression for large language models. *Transactions of the Association for Computational Linguistics*, 12:1517–1532, 2024. doi: 10.1162/tacl-a.00716. URL <https://aclanthology.org/2024.tacl-1.83/>.
- Dao, T. Flashattention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=mZn2Xyh9Ec>.
- Eyuboglu, S., Ehrlich, R. S., Arora, S., Guha, N., Zinsley, D., Liu, E. R., Rudra, A., Zou, J., Mirhoseini, A., and Re, C. Cartridges: Lightweight and general-purpose long context representations via self-study. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=0k5w800SNg>.
- Feng, Y., Lv, J., Cao, Y., Xie, X., and Zhou, S. K. Ada-KV: Optimizing KV cache eviction by adaptive budget allocation for efficient LLM inference. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=tcisuhGsQZ>.

- Fu, T., Gao, S., Zhao, X., rong Wen, J., and Yan, R. Learning towards conversational ai: A survey. *AI Open*, 3:14–28, 2022. ISSN 2666-6510. doi: <https://doi.org/10.1016/j.aiopen.2022.02.001>. URL <https://www.sciencedirect.com/science/article/pii/S2666651022000079>.
- Galley, M., McKeown, K. R., Fosler-Lussier, E., and Jing, H. Discourse segmentation of multi-party conversation. In *Proceedings of the 41st Annual Meeting of the Association for Computational Linguistics*, pp. 562–569, Sapporo, Japan, July 2003. Association for Computational Linguistics. doi: 10.3115/1075096.1075167. URL <https://aclanthology.org/P03-1071/>.
- Gliwa, B., Mochol, I., Biesek, M., and Wawer, A. SAM-Sum corpus: A human-annotated dialogue dataset for abstractive summarization. In Wang, L., Cheung, J. C. K., Carenini, G., and Liu, F. (eds.), *Proceedings of the 2nd Workshop on New Frontiers in Summarization*, pp. 70–79, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-5409. URL <https://aclanthology.org/D19-5409/>.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Hooper, C. R. C., Kim, S., Mohammadzadeh, H., Maheswaran, M., Zhao, S., Paik, J., Mahoney, M. W., Keutzer, K., and Gholami, A. Squeezed attention: Accelerating long context length LLM inference. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 32631–32652, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1568. URL <https://aclanthology.org/2025.acl-long.1568/>.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mistral 7b, 2023. URL <https://arxiv.org/abs/2310.06825>.
- Jiang, B., Yuan, Y., Shen, M., Hao, Z., Xu, Z., Chen, Z., Liu, Z., Vijjini, A. R., He, J., Yu, H., Poovendran, R., Wornell, G., Ungar, L., Roth, D., Chen, S., and Taylor, C. J. Personamem-v2: Towards personalized intelligence via learning implicit user personas and agentic memory, 2025. URL <https://arxiv.org/abs/2512.06688>.
- Joty, S., Carenini, G., and Ng, R. T. Topic segmentation and labeling in asynchronous conversations. *Journal of Artificial Intelligence Research*, 47:521–573, July 2013. ISSN 1076-9757. doi: 10.1613/jair.3940. URL <http://dx.doi.org/10.1613/jair.3940>.
- Kim, J.-H., Kim, J., Kwon, S., Lee, J. W., Yun, S., and Song, H. O. KVzip: Query-agnostic KV cache compression with context reconstruction. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=JFygzwx8SJ>.
- Kim, M., Shim, K., Choi, J., and Chang, S. InfiniPot: Infinite context processing on memory-constrained LLMs. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 16046–16060, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.897. URL <https://aclanthology.org/2024.emnlp-main.897/>.
- Kryscinski, W., Rajani, N., Agarwal, D., Xiong, C., and Radev, D. BOOKSUM: A collection of datasets for long-form narrative summarization. In Goldberg, Y., Kozareva, Z., and Zhang, Y. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2022*, pp. 6536–6558, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.findings-emnlp.488. URL <https://aclanthology.org/2022.findings-emnlp.488/>.
- Lee, D.-H., Maharana, A., Pujara, J., Ren, X., and Barbieri, F. Realtalk: A 21-day real-world dataset for long-term conversation. *arXiv preprint arXiv:2502.13270*, 2025.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive nlp tasks. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 9459–9474. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.
- Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. SnapKV: LLM knows what you are looking for before generation. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=poE54GOq2l>.

- Liu, G., Li, C., Zhao, J., Zhang, C., and Guo, M. Clusterv: Manipulating llm kv cache in semantic space for recallable compression. In *Proceedings of the 62nd Annual ACM/IEEE Design Automation Conference, DAC '25*. IEEE Press, 2025a. ISBN 9798331503048. doi: 10.1109/DAC63849.2025.11132479. URL <https://doi.org/10.1109/DAC63849.2025.11132479>.
- Liu, T., Xu, C., and McAuley, J. Repobench: Benchmarking repository-level code auto-completion systems. In *The Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=pPjZIOuQuF>.
- Liu, X., Chen, H., Hu, X., and Chu, X. FlowKV: Enhancing multi-turn conversational coherence in LLMs via isolated key-value cache management. In *First Workshop on Multi-Turn Interactions in Large Language Models*, 2025b. URL <https://openreview.net/forum?id=rZumUlowkr>.
- Liu, Z., Zhao, C., Iandola, F., Lai, C., Tian, Y., Fedorov, I., Xiong, Y., Chang, E., Shi, Y., Krishnamoorthi, R., Lai, L., and Chandra, V. MobileLLM: Optimizing sub-billion parameter language models for on-device use cases. In *Forty-first International Conference on Machine Learning*, 2024b. URL <https://openreview.net/forum?id=EIGbXbxcUQ>.
- Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., and Fang, Y. Evaluating very long-term conversational memory of LLM agents. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13851–13870, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.747. URL <https://aclanthology.org/2024.acl-long.747/>.
- Mao, Y., Wang, Q., Ester, M., and Li, K. Icecache: Memory-efficient KV-cache management for long-sequence LLMs. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=yHxSKM9kdr>.
- Meta. The llama 4 herd: The beginning of a new era of natively multimodal ai innovation. <https://ai.meta.com/blog/llama-4-multimodal-intelligence>, 2025. Accessed: 2025-01-25.
- NVIDIA. Kv-cache compression leaderboard. <https://huggingface.co/spaces/nvidia/kvpress-leaderboard>, 2025. Accessed: 2025-09-01.
- OpenAI. Gpt-4 technical report, 2024. URL <https://arxiv.org/abs/2303.08774>.
- Pan, Z., Wu, Q., Jiang, H., Luo, X., Cheng, H., Li, D., Yang, Y., Lin, C.-Y., Zhao, H. V., Qiu, L., and Gao, J. Secom: On memory construction and retrieval for personalized conversational agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=xKDZAW0He3>.
- Park, J., Jones, D., Morse, M. J., Goel, R., Lee, M., and Lott, C. Keydiff: Key similarity-based KV cache eviction for long-context LLM inference in resource-constrained environments. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=uBaFH7aQnC>.
- Qwen, :, Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Raedt, M., Godin, F., Develder, C., and Demeester, T. Revisiting clustering for efficient unsupervised dialogue structure induction. *Applied Intelligence*, 54:1–28, 04 2024. doi: 10.1007/s10489-024-05455-5.
- Reid, M., Savinov, N., Teplyashin, D., Lepikhin, D., Lillcrap, T., Alayrac, J.-b., Soricut, R., Lazaridou, A., Firat, O., Schrittwieser, J., et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- Tan, Z., Yan, J., Hsu, I.-H., Han, R., Wang, Z., Le, L., Song, Y., Chen, Y., Palangi, H., Lee, G., Iyer, A. R., Chen, T., Liu, H., Lee, C.-Y., and Pfister, T. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8416–8439, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.413. URL <https://aclanthology.org/2025.acl-long.413/>.
- Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., and Han, S. QUEST: Query-aware sparsity for efficient long-context LLM inference. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=KzACYw0MTV>.

- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models, 2023. URL <https://arxiv.org/abs/2302.13971>.
- Uddin, M. N., Shubham, K., Blanco, E., Baral, C., and Wang, G. From recall to forgetting: Benchmarking long-term memory for personalized agents, 2026. URL <https://arxiv.org/abs/2604.20006>.
- Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., and Yu, D. Longmemeval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations*, 2025a. URL <https://openreview.net/forum?id=pZiyCaVuti>.
- Wu, W., Wang, Y., Xiao, G., Peng, H., and Fu, Y. Retrieval head mechanistically explains long-context factuality. In *The Thirteenth International Conference on Learning Representations*, 2025b. URL <https://openreview.net/forum?id=EytBpUGB1Z>.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=NG7sS51zVF>.
- Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., and Zhang, Y. A-mem: Agentic memory for LLM agents. In *The Thirtieth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=Fim0M8gcct>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.
- Yang, S., Lee, J., Bang, J., Shim, K., Kim, M., and Chang, S. Learning contextual retrieval for robust conversational search. In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V. (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 11991–12003, Suzhou, China, November 2025b. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.602. URL <https://aclanthology.org/2025.emnlp-main.602/>.
- Zhang, Y., Li, M., Long, D., Zhang, X., Lin, H., Yang, B., Xie, P., Yang, A., Liu, D., Lin, J., Huang, F., and Zhou, J. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.
- Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Re, C., Barrett, C., Wang, Z., and Chen, B. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=RkRrPp7GKO>.
- Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=VqkAKQibpq>.
- Zhong, W., Guo, L., Gao, Q., Ye, H., and Wang, Y. Memorybank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17): 19724–19731, Mar. 2024. doi: 10.1609/aaai.v38i17.29946. URL <https://ojs.aaai.org/index.php/AAAI/article/view/29946>.
- Zhu, Y., Tang, Z., Liu, X., Li, A., Li, B., Chu, X., and Han, B. OracleKV: Oracle guidance for question-independent KV cache compression. In *ICML 2025 Workshop on Long-Context Foundation Models*, 2025. URL <https://openreview.net/forum?id=KHM2YOGgX9>.

A. Experimental Details

A.1. Dataset

We evaluate EPICACHE on three LongConvQA benchmarks: Realtalk (Lee et al., 2025), LoCoMo (Maharana et al., 2024), and LongMemEval (Wu et al., 2025a). Three benchmarks follow the LongConvQA formulation in Section 2.1, where a long conversational history $\mathcal{H}$ is provided and the model is required to answer a sequence of queries $\mathcal{Q} = q_1, \dots, q_{N_q}$ grounded in dialogue history. This formulation evaluates the answer accuracy of LLMs in a *multi-turn conversation*.

Realtalk. Realtalk (Lee et al., 2025) is a real-world dataset of 10 long-term conversations, where pairs of participants engaged in daily messaging for 16-21 days. Unlike LLM-simulated corpora such as LoCoMo, Realtalk captures natural dialogue including typos, abbreviations, asynchronous response gaps, and consecutive messages, while also reflecting diverse emotional expressions and shifts in persona consistency.

For evaluation, the dataset provides annotated memory probing questions across three subtasks—multi-hop, temporal reasoning, and commonsense—requiring models to recall and reason over extended histories. Following the original setup, we adopt GPT-based scoring (gpt-4o-mini-2024-07-18) to assess open-ended generation.

LoCoMo. LoCoMo (Maharana et al., 2024) is a benchmark of long-term conversations, created through a human-machine pipeline where LLM-based agents generate dialogues grounded in distinct personas and temporal event graphs, and human annotators refine them for long-range consistency. The dataset consists of 10 conversations, each spanning up to 35 sessions with around 300 turns. The QA benchmark is divided into five subtasks: (i) Single-hop, (ii) Multi-hop, (iii) Temporal reasoning, (iv) Open-domain knowledge, and (v) Adversarial. Evaluation is conducted with open-ended generation with F1 score.

We exclude the *adversarial* subtask for the following reason. This task tests whether a model can recognize unanswerable questions by choosing between a plausible but incorrect answer and a “no such information” response. Under KV cache compression, however, models frequently over-predict the latter, which leads to spuriously high scores. For example, with LLaMA-3.2-3B the adversarial score is only 12.11 under full KV, yet jumps to 49.78 with 4K KVzip compression—an increase that reflects bias rather than genuine improvement.

This behavior contrasts with other subtasks such as temporal or multi-hop reasoning, where compressed caches consistently degrade performance. Because open-source models already struggle on adversarial questions (Maharana et al., 2024), reporting these inflated numbers would give a misleading evaluation of answer quality under compressed KV cache. We therefore omit adversarial results from our main evaluation and defer a more thorough study of unanswerability detection under compression to future work.

LongMemEval. LongMemEval (Wu et al., 2025a) benchmarks long-term memory in user-assistant interactions with five core abilities—information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention—through seven question types (single-session user/assistant/preference, two-hop, multi-session synthesis, knowledge update, temporal reasoning, and abstention). A key property is its *length-configurable* chat histories: the benchmark provides standardized settings with extremely long contexts (e.g., up to 1.5M tokens), designed as controlled stress tests of memory and retrieval mechanisms. We follow the open-ended generation setup and report F1 scores for this dataset.

To align LongMemEval with the LongConvQA formulation in Section 2.1, we utilize the custom session stacking provided by LongMemEval³ to build coherent long conversations from user-LLM. Using this feature, we construct evaluation sets while preserving the original distribution of all question types. Specifically, we sample QA pairs according to the benchmark’s task-type proportions, retrieve the corresponding evidence conversation sessions, and assemble them into chronologically consistent histories. We then evaluate models at context lengths of 20K, 40K, 60K, 80K, and 100K tokens. This design allows us to test KV cache compression under scalable memory budgets.

A.2. KV Cache Compression Baseline Setup

We adapt existing KV cache compression methods to the block prefill setting for a fair comparison with our approach. The baselines include both static retention and attention-based eviction strategies, as well as similarity-based selection.

³<https://github.com/xiaowu0162/LongMemEval>

Algorithm 1 EpiCache with Layer-wise Budget Allocation Pseudo Code

Require: $\mathcal{H}$ (history, N_u turns), f_{embed} , w_{embed} , E , M ; f_{LM} (L layers, H heads); masks $\{\mathcal{M}, \mathcal{M}'\}$; sharpness α ; calibration batch $\mathcal{B}$ with $|\mathcal{B}| = 1$; patched prompt $\mathcal{P}_e$ (built from medoid segments, see stage A1)

Ensure: Episodic caches $\mathbb{B} = \{C_{\text{KV}}^{(1)}, \dots, C_{\text{KV}}^{(E)}\}$, centroids $\{\mathbf{c}_e\}_{e=1}^E$, and layer budgets $\{M_\ell^{\text{alloc}}\}_{\ell=1}^L$

Phase A: Clustering and Prefill
A1. Conversation Segment & Clustering (Offline)

- 1: Partition $\mathcal{H}$ into $K = \lceil N_u / w_{\text{embed}} \rceil$ segments $\{S_k\}_{k=1}^K$ and encode $\mathbf{e}_k = f_{\text{embed}}(S_k)$.
- 2: Run K-Means clustering on $\{\mathbf{e}_k\}$ to obtain $\{\mathcal{E}_e\}_{e=1}^E$.
- 3: **for** $e = 1$ **to** E **do**
- 4: $\mathbf{c}_e \leftarrow \frac{1}{|\mathcal{E}_e|} \sum_{S_k \in \mathcal{E}_e} \mathbf{e}_k$; $S_{\text{medoid}}^{(e)} \leftarrow \arg \max_{S_k \in \mathcal{E}_e} \cos(\mathbf{e}_k, \mathbf{c}_e)$
- 5: Build patched prompt $\mathcal{P}_e$ by concatenating utterances of $S_{\text{medoid}}^{(e)}$.
- 6: **end for**

A2. Measure layer sensitivity & allocate KV budgets.

- 7: **for each** $x \in \mathcal{B}$ **do**
- 8: $K_{\text{full}}^\ell(x) \leftarrow f_{\text{LM}}(x, \mathcal{M})W_K^\ell$; $K_{\text{block}}^\ell(x) \leftarrow f_{\text{LM}}(x, \mathcal{M}')W_K^\ell$ **for** $\ell = 1:L$
- 9: $\sigma_\ell(x) \leftarrow \frac{1}{HN} \sum_{h=1}^H \sum_{i=1}^N \cos(k_{\text{full},i}^{(\ell,h)}(x), k_{\text{block},i}^{(\ell,h)}(x))$
- 10: **end for**
- 11: $\sigma_\ell \leftarrow \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} \sigma_\ell(x)$; $s_\ell \leftarrow 1 - \sigma_\ell$
- 12: $w_\ell \leftarrow \frac{s_\ell^\alpha}{\sum_{j=1}^L s_j^\alpha}$; $M_\ell^{\text{alloc}} \leftarrow (L \cdot M) w_\ell$

A3. Build episodic KV caches.

- 13: **for** $e = 1$ **to** E **do**
- 14: Block-wise prefill over $\mathcal{H}$, appending $\mathcal{P}_e$ to each block of M_{block} tokens.
- 15: Compute scores w.r.t. $\mathcal{P}_e$ with Equation (3) and retain the top M tokens.
- 16: $C_{\text{KV}}^{(e)} \leftarrow$ compressed cache for episode e .
- 17: **end for**
- 18: $\mathbb{B} \leftarrow \{C_{\text{KV}}^{(1)}, \dots, C_{\text{KV}}^{(E)}\}$.

Phase B: Online decoding

- 19: For query q_i : $\mathbf{q}_i \leftarrow f_{\text{embed}}(q_i)$; $\mathbf{e}^\dagger \leftarrow \arg \max_e \cos(\mathbf{q}_i, \mathbf{c}_e)$
- 20: Retrieve $C_{\text{KV}}^{(\mathbf{e}^\dagger)}$ and generate with compressed cache: $f_{\text{LM}}(q_i \mid C_{\text{KV}}^{(\mathbf{e}^\dagger)})$.

StreamingLLM. Following StreamingLLM (Xiao et al., 2024), we retain a fixed number of sink and recent tokens throughout block prefill. Specifically, we fix the number of sink tokens to 128 for all models, while the remaining budget $M - 128$ is assigned to the most recent tokens.

SnapKV. We adapt SnapKV (Li et al., 2024) to the block prefill setting, where future queries are not accessible in LongConvQA. Following the original design, we use the window tokens—in our case the last 64 tokens of each block—as the patched prompt, and then apply the scoring function in Equation (3). Tokens with the highest attention relevance to this patched prompt are retained.

KVzip. We adapt KVzip (Kim et al., 2026) to block prefill by treating the entire block of tokens as the patched prompt. At each block boundary, we append a repetition instruction (e.g., "Repeat the part of the previous context exactly") followed by the full block tokens, and then apply the patched-prompt scoring method.

InfiniPot. We adopt the InfiniPot (Kim et al., 2024) by employing a general-purpose patched prompt designed to highlight globally important content. Specifically, we append the instruction "Summarize the previous context highlighting the most important parts." at the end of each block and compute scores according to Equation (3). This encourages selection of semantically informative tokens across the block.

KeyDiff. KeyDiff (Park et al., 2025) is KV cache eviction method for block prefill. For each block, it constructs an anchor key by averaging the key states of all tokens, and computes the dissimilarity score of each token as the negative cosine similarity between its key state and the anchor. These scores are then used to guide eviction. Following the original implementation, we evaluate $M_{\text{block}} \in 128, 512, 1024, 2048$, which includes the default setting of 128, and report results using the configuration that achieved the best performance.

To ensure fair comparison, all attention-based methods (SnapKV, InfiniPot, KVzip, EpiCache) use the same scoring

formulation from Equation (3), and all eviction methods are combined with head-wise non-uniform token selection as suggested by Feng et al. (2026).

A.3. EPICACHE Setup

Overall Process. We provide the complete procedure of EPICACHE in Algorithm 1. The framework consists of two phases. In **Phase A**, the conversation history is clustered into topical episodes and a compressed KV cache is prepared for each episode. In **Phase B**, online queries are answered by retrieving the most relevant episodic cache.

In *Phase A1*, we segment the conversation history, embed each segment, and cluster them into E episodes. This step can be performed offline, and the cost of segment encoding and K-means clustering is negligible (under a minute). Each episode is represented by its centroid and a medoid segment that serves as a patched prompt. In *Phase A2*, we measure per-layer sensitivity by comparing Key states under full and block-wise prefill masks, and allocate layer-wise budgets proportionally using the sharpness hyper-parameter α . In *Phase A3*, we construct episodic KV caches by performing block-wise prefill with the patched prompt appended, and then compress the resulting caches according to the allocated budgets from Phase A2. Although prefill must be repeated for every episode, the peak memory remains flat during this process (see Figure 1c), making it practical for constrained-memory environments.

In Phase B, when a new query arrives, we embed the query and compute its similarity to the episode centroids. The query is then routed to the most relevant episodic cache, which is loaded for decoding. If the same cache is selected as in the previous turn, no additional retrieval is required since the cache remains resident, further reducing overhead.

Detailed Settings For segment construction, we set the embedding window size w_{embed} to 4, selected from 2, 4, 8. To cluster segments into episodes, we apply the standard K-means algorithm with k-means++ initialization (Arthur & Vassilvitskii, 2007). This offline segmentation and clustering stage completes within one minute, incurring negligible overhead.

For sensitivity-aware budget allocation, we estimate per-layer weights using a single randomly sampled long document from the BookSum (Kryscinski et al., 2022) dataset, chosen to avoid bias from any specific conversational dataset. By performing two forward passes—one with the full causal mask and one with the block-wise prefill mask—we measure layer-wise deviations and compute allocation weights. Because only one sample is used, the overhead of this calibration step is negligible.

In block prefill, the cache always maintains size M : as conversation segments are added in blocks of M_{block} , the cache can temporarily grow up to $M + M_{\text{block}}$ entries, after which eviction is applied to reduce it back to M . A larger M_{block} enables the model to cover the entire conversation more quickly but increases the temporary peak memory footprint, while a smaller M_{block} lowers peak memory at the cost of slower coverage. We set $M_{\text{block}} \in \{128, 512, 1024, 2048\}$ to balance this trade-off.

B. Further Analysis

B.1. Conversation Clustering Analysis

In this section, we provide qualitative examples of conversation clustering to illustrate how episodic structures emerge in practice. Conversation histories are divided into segments of $w_{\text{embed}} = 4$ utterances, which are then embedded using Qwen3-0.6B (Zhang et al., 2025). Segment embeddings are clustered with K-Means, and the resulting clusters are visualized in two dimensions via t-SNE, as shown in Figure A1 (a).

For each cluster, we further present representative medoid segments in Figure A1 (b). These examples demonstrate that the clustering procedure consistently groups segments into coherent topical episodes, such as games, movies, literature, or weather. The medoid samples highlight the interpretability of each episode and indicate how such episode-level partitioning can serve as the basis for episodic KV cache compression.

B.2. block-wise prefill Sensitivity Analysis

We analyze layer-wise deviations under block prefill by comparing multiple internal states—Key, Value, and layer outputs—across Transformer layers computed with the full causal mask ($\mathcal{M}$) and the block mask ($\mathcal{M}'$). To this end, we forward LoCoMo (Maharana et al., 2024) conversation history samples under both masks and plot the resulting internal

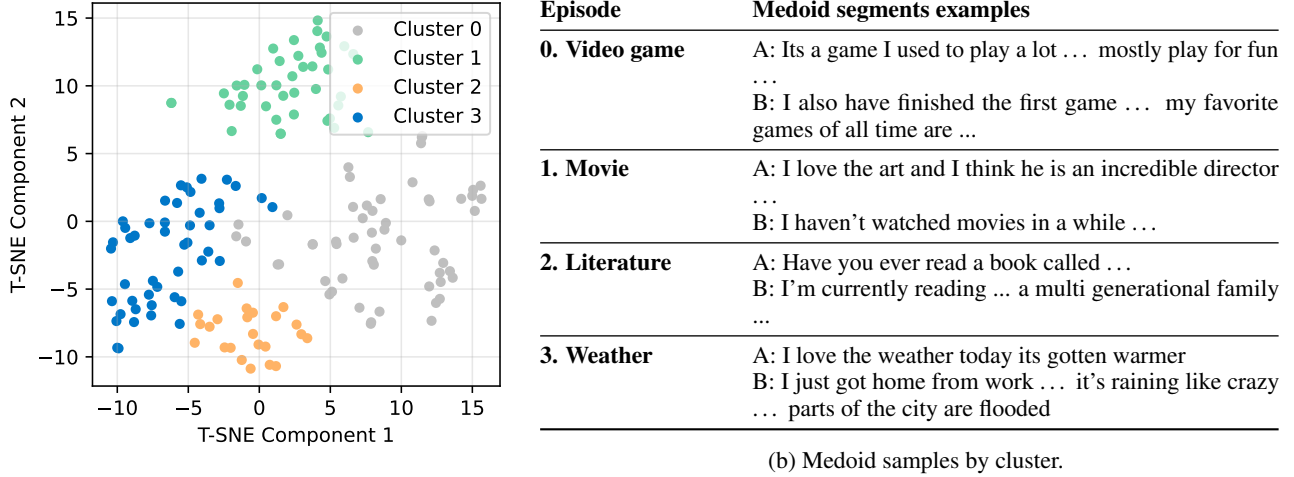


Figure A1. **Episodic clustering of conversation segments.** (a) t-SNE visualization of conversation clustering. (Silhouette score=0.28) (b) Medoid segments illustrate coherent topics per cluster.

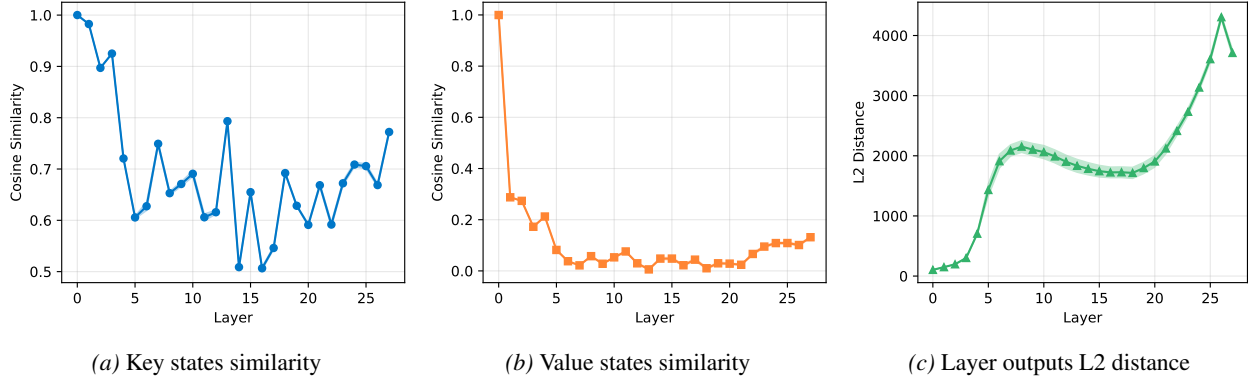


Figure A2. **Layer-wise Sensitivity Analysis.** layer-wise deviation results under the full and block masks using Qwen2.5-7B on LoCoMo conversation history. Key and Value states are measured by cosine similarity, while hidden states at layer outputs are measured by L2 distance; shaded regions indicate variance across input samples.

states differences across layers, as defined in Equations (8) and (9). Specifically, each plot reports cosine similarity of Key and Value states, and L2 distance of layer outputs, respectively.

We find that Value states (Figure A2b) exhibit consistently low similarity across layers, offering little discriminative trend. Layer outputs, measured by L2 distance (Figure A2c), show a monotonic error accumulation pattern rather than meaningful variation. In contrast, Key states (Figure A2a) provide clear differentiation across layers. This observation motivates our use of Key state deviation as the sensitivity measure for budget allocation, as discussed in Section 3.2. Further analysis of why these trends differ across Key, Value, and layer-wise output representations is left for future work.

C. Additional Experimental Results

C.1. Comprehensive LongConvQA Results

Figure A4 shows the comprehensive LongConvQA evaluation results, comparing EPICACHE against baselines (Xiao et al., 2024; Li et al., 2024; Kim et al., 2024; Park et al., 2025; Kim et al., 2026) across multiple cache budgets and four model variants, LLaMA3.2-3B, LLaMA3.1-8B, Qwen2.5-3B, and Qwen2.5-7B. The corresponding numerical results underlying this figure are reported in Table A4 for Realtalk and LoCoMo with LLaMA3.2-3B and LLaMA3.1-8B, in Table A5 for Qwen2.5-3B and 7B, and in Table A6 for LongMemEval across all four models.

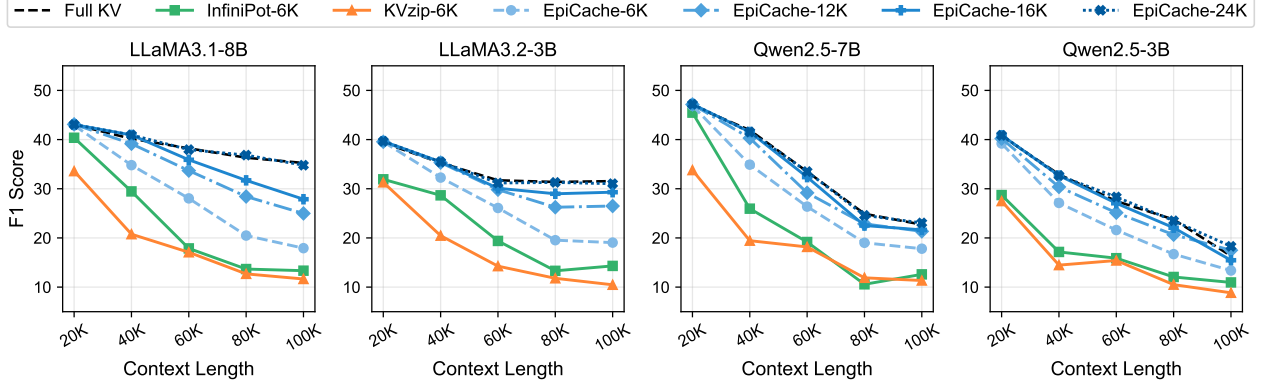


Figure A3. **Memory Scalability up to 100K Context.** Conversation histories between user and LLM-based assistant scaled to 100K tokens across four LLMs with LongMemEval. Comparison of InfiniPot and KVzip ($M=6K$) with EpiCACHe (4 episodes, $M=6K-24K$).

C.2. Memory Scalability Evaluation.

Figure A3 evaluates LongConvQA under extended conversation lengths, scaling up to 100K tokens.⁴ Open-source LLMs exhibit declining QA performance as context length grows to 100K, as reported in (Wu et al., 2025a), and the performance gap between full KV and baseline methods (KVzip, InfiniPot) becomes increasingly pronounced. EpiCACHe delivers higher accuracy than baselines at the same memory budget across all context lengths, and as the KV cache budget increases, its accuracy steadily approaches full KV, demonstrating the memory scalability of our approach.

D. Future Work

Several promising directions remain for future work. On the episode construction side, the quality of episodic boundaries depends heavily on the underlying segment representations: more effective conversational embeddings that capture turn-level context and inter-utterance dependencies could yield more coherent episode construction (Yang et al., 2025b), representing an improvement to our clustering and scoring pipeline. Complementarily, extending EpiCACHe to adaptively determine the optimal number of episodes (Raedt et al., 2024) could further improve scalability across dialogue lengths and domains.

Beyond improvements to episodic construction, long-term conversational QA surfaces richer memory management challenges: tracking implicit user preferences, handling information updates and deletions over time, and maintaining consistent persona across extended interactions (Jiang et al., 2025; Uddin et al., 2026). Finally, combining text-level memory organization (Tan et al., 2025; Pan et al., 2025; Xu et al., 2026) with efficient KV cache compression presents a promising direction toward accurate and efficient conversational agents deployable on resource-constrained devices.

⁴LongMemEval (Wu et al., 2025a) supports stacking conversation sessions with associated QA pairs, allowing conversation histories to be constructed at custom lengths.

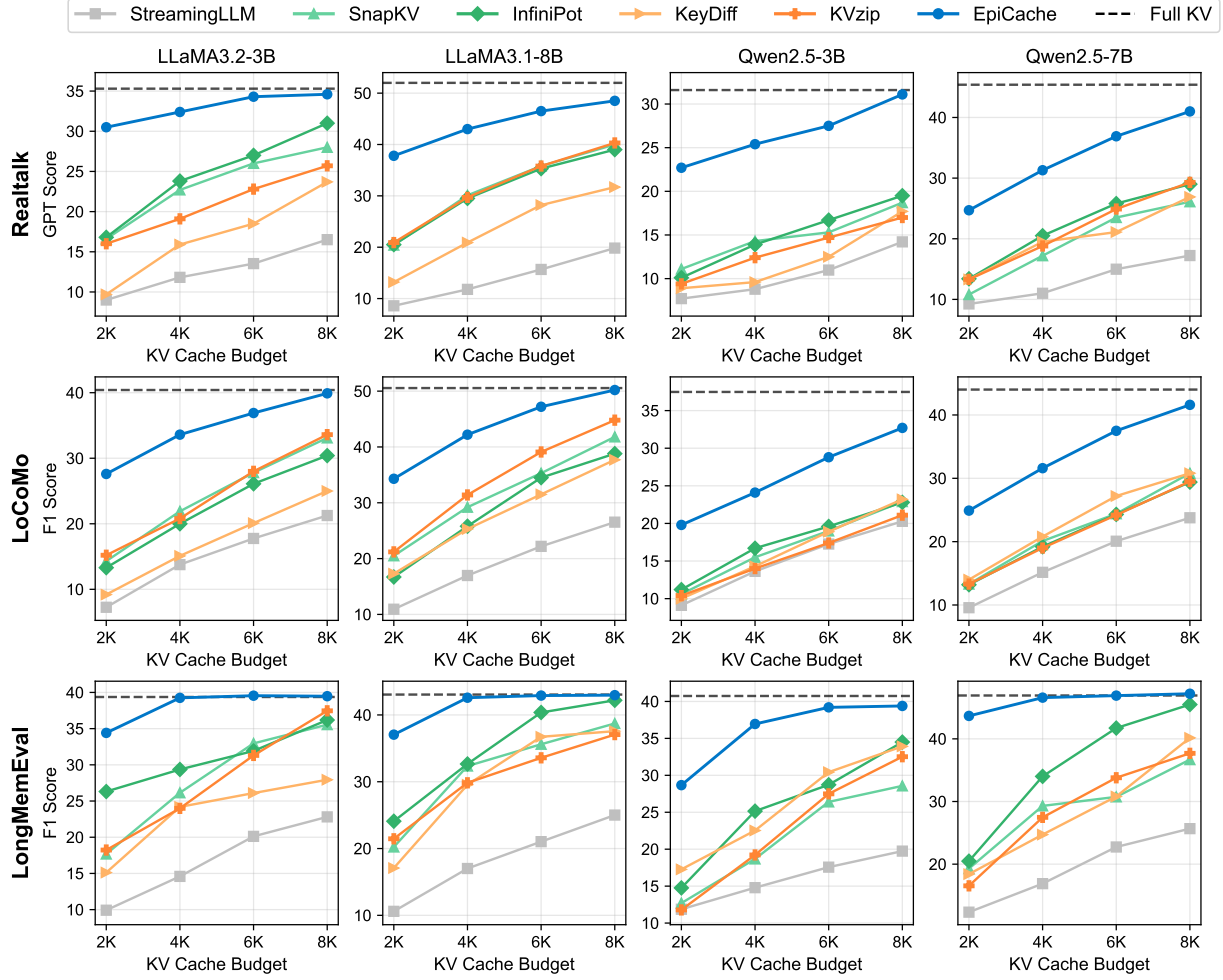


Figure A4. **Evaluation with Various Models** results with Realtalk, LoCoMo and LongMemEval. Evaluated with fixed KV cache budgets $M \in \{2K, 4K, 6K, 8K\}$ across four different models. The number of episodes (clusters) fixed to $E=4$ in all experiments.

LLaMA3.2-3B		LoCoMo (Full KV length: 21.8K)					Realtalk (Full KV length: 26.4K)			
Method	M	Multi-hop	Temporal	Open-domain	Single-hop	Avg	Multi-hop	Temporal	Common	Avg
Full KV	–	36.0	15.1	13.2	54.5	40.3	39.0	30.8	38.2	35.3
SnapKV (Li et al., 2024)	2K	17.3	3.7	17.3	17.6	14.3	20.1	10.5	25.0	16.6
	4K	23.1	6.0	11.5	28.7	21.9	28.5	14.7	30.0	22.7
	6K	27.3	9.9	10.6	36.8	27.8	31.7	18.2	33.2	26.0
	8K	31.2	11.9	11.7	44.3	33.1	34.1	20.3	33.7	28.0
InfiniPot (Kim et al., 2024)	2K	15.9	7.2	10.1	15.0	13.3	22.1	9.4	24.3	16.8
	4K	21.3	12.0	10.0	23.7	20.0	29.2	16.0	31.9	23.8
	6K	25.6	17.0	11.9	31.3	26.1	32.8	19.7	32.4	27.0
	8K	28.6	20.5	11.4	36.9	30.4	35.6	24.5	36.9	31.0
KeyDiff (Park et al., 2025)	2K	10.7	3.7	11.6	11.2	9.2	10.4	5.6	19.5	9.7
	4K	15.7	8.3	11.5	17.9	15.1	16.8	14.2	18.2	15.9
	6K	20.7	11.9	13.4	23.8	20.1	20.6	14.5	24.8	18.5
	8K	23.7	15.7	14.7	30.2	25.0	24.8	20.7	29.7	23.7
KVzip (Kim et al., 2026)	2K	22.0	4.5	12.0	17.3	15.2	18.7	9.5	27.9	16.0
	4K	21.9	13.3	10.0	24.5	20.8	25.0	10.1	29.4	19.1
	6K	28.1	16.5	12.0	34.2	28.0	26.1	15.2	36.3	22.8
	8K	31.2	21.8	11.8	41.5	33.6	30.7	18.1	34.5	25.7
EpiCache	2K	29.3	15.9	13.7	33.1	27.6	34.7	23.0	41.3	30.5
	4K	30.4	19.7	14.2	42.3	33.6	37.4	24.7	41.0	32.4
	6K	33.7	22.2	12.1	46.4	36.9	39.9	25.6	44.2	34.3
	8K	33.9	23.9	12.7	48.2	38.3	40.0	26.6	42.9	34.6

LLaMA3.1-8B		LoCoMo (Full KV length: 21.8K)					Realtalk (Full KV length: 26.4K)			
Method	M	Multi-hop	Temporal	Open-domain	Single-hop	Avg	Multi-hop	Temporal	Common	Avg
Full KV	–	43.1	22.7	17.2	67.4	50.5	49.2	55.9	48.4	52.0
SnapKV (Li et al., 2024)	2K	23.4	6.6	14.3	25.5	20.5	20.2	15.8	34.3	20.4
	4K	30.8	10.9	13.6	37.3	29.2	28.7	26.6	44.4	30.1
	6K	33.3	13.8	13.3	46.8	35.3	37.3	32.2	42.3	35.8
	8K	37.7	18.1	14.2	55.5	41.8	41.5	36.8	45.2	40.0
InfiniPot (Kim et al., 2024)	2K	15.2	12.3	10.2	19.6	16.7	21.2	14.9	35.0	20.5
	4K	23.0	22.1	13.9	29.5	25.8	30.0	25.8	39.1	29.5
	6K	29.8	29.6	13.9	40.1	34.5	34.4	33.3	43.4	35.3
	8K	33.1	33.8	13.8	45.5	38.8	38.3	38.2	43.4	39.0
KeyDiff (Park et al., 2025)	2K	18.7	4.4	17.7	21.8	17.3	13.0	8.2	28.8	13.2
	4K	23.3	14.1	13.8	31.5	25.3	21.9	16.5	30.9	20.9
	6K	27.5	18.6	14.5	39.7	31.5	29.6	25.3	33.3	28.2
	8K	33.0	25.8	16.4	46.2	37.7	33.0	29.9	33.3	31.7
KVzip (Kim et al., 2026)	2K	24.4	8.4	22.8	24.7	21.2	20.6	16.6	34.6	20.9
	4K	28.8	24.3	15.3	36.8	31.4	29.5	26.4	40.0	29.7
	6K	32.8	31.0	13.3	47.3	39.1	36.4	33.4	41.5	35.8
	8K	37.2	35.0	13.1	54.6	44.8	38.8	41.3	41.1	40.3
EpiCache	2K	33.2	26.3	14.2	43.5	36.3	37.9	35.2	45.2	37.8
	4K	37.7	33.8	15.9	55.8	45.4	39.6	44.9	46.7	43.0
	6K	38.2	36.4	16.4	58.2	47.3	42.3	50.5	35.2	46.5
	8K	38.4	37.5	17.4	62.7	50.2	44.2	50.9	46.5	47.5

Table A4. LongConvQA (LoCoMo and Realtalk) Evaluation: Comparison of different KV cache compression methods under block-
prefill with LLaMA series models.

Qwen2.5-3B		LoCoMo (Full KV length: 21.9K)					Realtalk (Full KV length: 26.6K)			
Method	M	Multi-hop	Temporal	Open-domain	Single-hop	Avg	Multi-hop	Temporal	Common	Avg
Full KV	–	33.2	22.9	12.3	49.1	38.4	32.7	28.0	39.6	31.6
SnapKV (Li et al., 2024)	2K	14.1	8.0	11.5	10.4	10.6	12.4	5.6	23.5	11.1
	4K	17.9	12.7	11.5	16.2	15.5	15.4	9.0	27.0	14.3
	6K	21.3	13.1	13.7	21.1	19.0	17.9	9.3	26.2	15.3
	8K	23.2	14.3	12.1	27.4	22.9	21.6	12.6	28.8	18.7
InfiniPot (Kim et al., 2024)	2K	12.4	16.1	12.6	8.8	11.2	8.6	6.1	26.3	10.1
	4K	18.2	19.7	10.2	15.7	16.7	15.4	8.9	24.3	13.9
	6K	20.0	20.0	13.9	19.9	19.6	17.5	12.0	28.2	16.7
	8K	23.9	18.8	13.2	25.1	22.8	23.9	11.4	31.1	19.5
KeyDiff (Park et al., 2025)	2K	10.4	14.2	13.6	7.8	10.0	3.9	9.0	23.5	8.9
	4K	14.0	15.8	12.0	14.1	14.3	9.2	6.0	21.6	9.6
	6K	19.6	16.4	9.2	20.8	18.9	11.5	10.0	22.6	12.5
	8K	21.6	17.3	7.4	27.8	23.2	18.1	15.1	24.4	17.7
KVzip (Kim et al., 2026)	2K	11.8	6.1	11.9	11.4	10.4	10.6	5.3	18.2	9.4
	4K	16.2	10.3	12.6	14.7	14.0	13.3	8.4	21.7	12.4
	6K	19.5	12.7	11.3	19.1	17.4	15.0	10.0	27.8	14.7
	8K	21.9	14.3	14.1	24.3	21.1	18.8	11.4	28.4	17.0
EpiCache	2K	23.6	7.6	13.4	23.8	19.8	25.6	13.2	37.7	22.0
	4K	27.1	10.7	11.4	29.7	24.1	30.0	17.1	36.9	25.4
	6K	27.3	13.3	10.8	37.2	28.8	32.6	19.7	36.4	27.5
	8K	31.3	17.0	10.0	41.7	32.7	33.9	25.0	41.1	31.1

Qwen2.5-7B		LoCoMo (Full KV length: 21.9K)					Realtalk (Full KV length: 26.6K)			
Method	M	Multi-hop	Temporal	Open-domain	Single-hop	Avg	Multi-hop	Temporal	Common	Avg
Full KV	–	36.2	19.2	16.6	59.3	44.1	38.7	52.3	43.4	45.3
SnapKV (Li et al., 2024)	2K	17.6	5.9	12.6	14.7	13.3	9.8	10.5	14.2	10.8
	4K	23.8	8.0	13.1	24.2	20.1	15.8	16.0	24.8	17.2
	6K	27.8	9.1	15.2	30.2	24.4	20.6	24.4	29.2	23.5
	8K	30.8	13.2	14.9	39.4	30.8	24.4	25.1	33.5	26.1
InfiniPot (Kim et al., 2024)	2K	15.1	14.2	12.0	12.4	13.2	10.4	15.7	15.2	13.4
	4K	19.2	20.0	10.8	19.9	19.2	17.1	22.7	23.7	20.5
	6K	23.4	23.7	15.0	26.0	24.3	21.6	30.0	25.5	25.8
	8K	27.7	14.0	14.0	32.7	29.4	26.3	29.4	31.5	29.0
KeyDiff (Park et al., 2025)	2K	12.8	16.7	12.4	13.6	14.0	9.8	15.0	18.4	13.3
	4K	18.9	22.0	13.9	21.8	20.8	12.0	19.9	26.3	19.5
	6K	26.3	25.2	14.3	29.8	27.2	15.7	26.1	21.1	21.1
	8K	28.3	23.7	15.9	36.4	30.8	20.3	32.7	28.0	26.9
KVzip (Kim et al., 2026)	2K	14.3	13.7	11.7	12.9	13.3	12.2	10.8	23.9	13.3
	4K	19.4	16.2	13.9	20.6	19.0	17.7	16.0	29.8	18.8
	6K	24.3	19.5	12.5	27.2	24.2	22.5	24.6	32.6	24.9
	8K	25.6	23.8	13.0	34.8	29.5	26.8	28.7	37.7	29.3
EpiCache	2K	26.4	15.4	13.2	29.3	24.9	24.1	21.3	36.6	24.7
	4K	29.0	20.9	14.1	38.5	31.6	31.1	29.3	37.5	31.3
	6K	32.6	24.6	15.1	46.5	37.5	33.0	39.4	40.6	36.9
	8K	32.6	28.1	15.3	52.7	41.6	33.8	46.9	43.6	41.0

Table A5. LongConvQA (LoCoMo and Realtalk) Evaluation: Comparison of different KV cache compression methods under block-prefill with Qwen series models.

Method	M	LLaMA3.2-3B								LLaMA3.1-8B							
		SH	TH	MS	TR-E	TR-I	KU	IP	Avg.	SH	TH	MS	TR-E	TR-I	KU	IP	Avg.
Full KV	21K	84.6	10.0	12.5	47.9	27.1	52.3	6.2	39.4	87.2	14.1	17.6	56.5	28.5	56.1	6.3	43.1
SnapKV (Li et al., 2024)	2K	26.9	0.8	1.8	26.6	17.4	31.4	6.1	17.7	35.6	3.8	3.3	29.8	23.3	25.4	8.8	20.2
	4K	40.0	5.3	2.4	40.2	20.4	48.5	6.5	26.1	63.3	9.7	3.9	45.8	24.4	45.8	10.8	32.4
	6K	54.5	5.3	15.8	37.1	23.0	58.7	6.3	33.0	65.1	8.3	9.0	49.4	30.4	50.5	11.8	35.6
	8K	67.1	7.9	12.4	37.1	27.1	56.6	7.1	35.6	74.3	13.1	13.6	53.8	25.9	52.5	10.9	38.7
InfiniPot (Kim et al., 2024)	2K	46.2	0.8	10.8	40.5	19.1	40.7	8.9	26.3	39.8	1.8	3.4	29.1	31.7	35.8	7.6	24.1
	4K	48.5	7.9	12.3	33.6	18.4	52.3	7.6	29.4	62.4	7.9	4.1	53.9	23.0	46.8	9.2	32.7
	6K	60.0	2.6	12.4	33.6	25.9	51.7	6.7	31.9	81.3	11.0	13.3	54.9	28.1	54.9	5.6	40.4
	8K	76.0	4.4	13.3	40.7	25.0	52.9	7.6	36.2	90.3	9.1	18.8	54.0	28.5	56.8	9.4	42.2
KeyDiff (Park et al., 2025)	2K	35.3	0.5	4.1	34.3	17.7	5.7	2.6	15.1	26.0	6.8	9.5	28.8	12.3	22.8	5.5	17.1
	4K	54.2	2.1	2.4	34.3	15.4	34.4	6.8	24.2	60.0	14.7	12.9	41.4	20.2	31.8	6.1	29.6
	6K	55.8	6.5	7.4	54.3	11.4	32.2	9.0	26.9	69.2	11.2	13.6	48.4	26.1	52.1	8.5	36.7
	8K	56.1	2.1	6.6	37.1	25.9	37.7	7.7	27.9	70.7	11.2	18.4	46.1	30.0	50.1	5.2	37.6
KVzip (Kim et al., 2026)	2K	30.8	0.0	1.8	30.9	15.2	30.3	7.6	18.2	33.8	7.5	8.0	36.0	19.4	27.2	9.1	21.5
	4K	44.7	2.6	6.5	37.1	15.0	37.8	7.4	24.0	56.6	9.7	6.7	35.3	24.1	42.7	12.4	29.8
	6K	58.1	5.3	12.4	37.1	21.3	50.8	6.2	31.3	61.3	11.8	6.9	42.7	29.4	47.5	11.3	33.6
	8K	73.5	7.9	12.5	40.7	26.2	59.1	6.9	37.5	73.6	14.4	7.7	48.6	26.9	51.4	6.2	37.1
EpiCACHE	2K	73.0	10.5	7.4	40.5	21.0	50.8	6.0	34.4	72.3	16.7	3.3	46.5	24.0	56.4	10.5	37.1
	4K	79.9	12.6	16.6	41.4	27.0	53.9	9.1	39.3	87.2	14.1	17.4	56.5	28.5	54.6	3.9	42.6
	6K	85.0	10.0	13.4	40.7	27.2	55.1	8.3	39.6	83.8	13.8	25.4	55.7	25.3	54.9	10.7	42.9
	8K	85.0	10.0	12.5	40.7	26.6	56.6	6.2	39.5	88.2	13.5	17.5	56.5	28.5	55.2	6.4	43.0

Method	M	Qwen2.5-3B								Qwen2.5-7B							
		SH	TH	MS	TR-E	TR-I	KU	IP	Avg.	SH	TH	MS	TR-E	TR-I	KU	IP	Avg.
Full KV	21K	80.8	14.0	15.0	50.2	23.7	59.0	9.1	40.7	88.6	39.7	35.1	32.9	35.4	47.9	12.9	46.9
SnapKV (Li et al., 2024)	2K	23.5	1.2	1.3	11.9	0.1	25.6	3.8	12.7	19.9	4.9	8.3	33.1	21.5	31.1	8.9	19.3
	4K	48.8	6.5	0.8	11.9	1.1	27.9	4.6	18.7	42.9	10.9	22.0	33.6	28.8	38.4	10.7	29.3
	6K	59.3	3.9	19.6	32.4	5.9	34.5	5.7	26.4	50.4	10.9	17.0	32.9	28.4	42.6	11.3	30.7
	8K	56.3	11.4	20.4	29.8	7.9	41.7	8.1	28.6	67.4	17.1	24.4	32.9	31.6	42.9	14.1	36.7
InfiniPot (Kim et al., 2024)	2K	31.0	1.2	11.9	22.6	5.4	17.5	4.9	14.8	29.8	1.6	2.0	33.3	23.1	32.2	11.8	20.5
	4K	46.2	5.3	16.5	33.3	19.4	31.5	6.8	25.2	44.7	22.6	25.3	36.4	36.7	41.6	10.3	34.0
	6K	55.9	4.8	10.6	32.4	25.7	40.1	6.1	28.7	66.1	26.7	32.9	36.4	40.5	48.1	11.7	41.7
	8K	70.2	6.1	15.7	35.9	27.3	47.3	7.7	34.5	78.6	34.0	33.6	32.9	41.4	50.1	13.1	45.5
KeyDiff (Park et al., 2025)	2K	12.6	14.0	6.3	44.1	11.1	27.6	3.3	17.3	18.1	8.2	9.4	31.1	14.3	31.5	11.7	18.4
	4K	30.5	11.4	11.1	34.5	18.9	33.0	5.7	22.5	43.1	5.8	9.9	37.1	20.5	32.7	10.7	24.7
	6K	51.6	15.3	21.1	43.7	20.5	35.4	9.6	30.4	57.5	13.0	13.9	34.2	26.8	38.0	9.9	30.8
	8K	62.6	22.6	15.4	30.7	22.0	47.5	7.8	33.9	69.6	23.6	32.9	39.1	25.6	50.2	11.9	40.2
KVzip (Kim et al., 2026)	2K	18.4	1.2	1.3	13.7	10.9	22.5	4.2	11.8	21.1	4.8	2.4	22.8	19.8	27.1	8.9	16.5
	4K	39.4	6.5	5.0	23.8	13.8	25.3	6.4	19.2	45.4	14.1	22.0	31.7	23.3	29.9	11.5	27.5
	6K	51.9	1.2	20.0	44.1	17.6	34.3	5.7	27.5	57.8	23.5	22.8	33.6	31.9	34.4	11.2	33.8
	8K	68.4	8.8	15.0	39.5	20.7	40.9	10.0	32.5	69.6	23.5	23.4	32.9	35.0	40.3	10.6	37.7
EpiCACHE	2K	52.6	3.5	10.0	32.4	22.9	46.7	6.7	28.7	70.4	36.9	31.1	42.4	35.9	48.5	12.4	43.7
	4K	74.4	14.0	11.0	46.7	17.6	55.9	9.1	37.0	83.6	38.3	29.8	37.6	49.6	41.3	12.8	46.6
	6K	77.3	16.7	15.4	46.7	22.0	55.3	10.5	39.2	86.1	38.3	33.1	32.9	39.9	48.8	12.7	46.9
	8K	77.3	16.7	15.0	46.7	22.8	55.9	10.1	39.4	86.0	38.3	33.1	37.6	40.5	47.3	12.9	47.2

Table A6. **LongConvQA (LongMemEval) Evaluation:** Evaluation results with Qwen and LLaMA series models under block prefill. SH = Single Hop, TH = Two Hop, MS = Multi-Session, TR-E = Temporal Reasoning (explicit), TR-I = Temporal Reasoning (implicit), KU = Knowledge Update. IP = Implicit Preference.